

Reconstructing the Exact LLM Inputs

How to read the benchmark questions from the repository's code and metadata

Repository: github.com/ericmedlock/MSAI-LLM-PERFORMANCE Guide version: 2026-07-11

ITCS 6881 — LLM Execution Architecture Benchmark

The one-paragraph answer

Every question the model ever sees is stored in plain text in the repository, in three composable layers: (1) the **task manifests** under `tasks/` hold each question's exact user-message text in the `prompt` field; (2) the **frozen system prompts** under `prompts/` hold the per-role instructions that precede it; (3) `config/config.yaml` pins the decoding parameters (temperature, seed, context, output budget). On top of that, every recorded run in `results/*.jsonl` carries a `raw_trace` of the messages *actually sent*, so any historical run can be audited without reconstruction. Nothing is generated at run time; reproduction is a matter of reading files, not re-running code.

1 Layer 1 — the task manifests (`tasks/`)

File	Contents
<code>manifest.json</code>	the frozen 15-item <i>baseline</i> suite (5 GSM8K / 5 HumanEval / 5 HotpotQA)
<code>frontier.v2.manifest.json</code>	the frozen 36-item <i>frontier</i> tier (12 AIME 2025 / 12 BigCodeBench-Hard / 12 MuSiQue open-book)
<code>frontier.manifest.json</code> , <code>frontier.external.manifest.json</code>	superseded candidate generations, kept for provenance

Each entry in a manifest's `tasks` array has these fields:

- `task_id` — the stable key every result row references (e.g. `fx2-mathA-001`).
- `prompt` — **the complete user-message text, verbatim.** What you read here is byte-for-byte what the model receives as the user turn — including the answer-format instruction line, and (for multihop items) the embedded context passages, and (for code items) the function signature and docstring to complete.
- `answer` — the gold answer for exact/normalized matching (`null` for code items, which grade by unit test instead).
- `grading` — for code items: `entry_point` (the function name) and `test` (the full test source; the grader executes `model code + test + check(entry_point)` in a subprocess).
- `source_id` — upstream identity, e.g. `aime25:0`, `bigcodebench-hard-v0.1.4:BigCodeBench/16`, `musique:2hop__123...` (with context), `gsm8k/main train[0]`.

- `id_status` — provenance verdict (`FROZEN_CALIBRATED_IN_BAND`, `VERIFIED_UPSTREAM_EXACT` / `_ADAPTED`, `PROJECT_AUTHORED`) — see `scripts/verify_task_ids.py` and the Amendment Log.
- `selection_reason` — why the item is in the suite (human-readable audit trail, including calibration history).
- `reference_solution` — (code items) the upstream canonical solution, used only to self-verify the test at build time; never shown to the model.

2 Layer 2 — the frozen system prompts (prompts/)

Each architecture role has its own version-controlled system prompt:

<code>prompts/monolithic/system.txt</code>	the single-shot solver
<code>prompts/agentive/executor_system.txt</code>	the agentive executor
<code>prompts/agentive/verifier_system.txt</code>	the agentive verifier
<code>prompts/swarm/peer_system.txt</code>	every swarm peer (identical for all three)
<code>prompts/judge/system.txt</code>	the LLM-as-judge (post-processing only)

These files are frozen; `prompts/README.md` documents the intentional role differences.

3 Layer 3 — message composition per architecture

The harness composes exactly two-role conversations (one system + one user message per call). From the backend sources (`backends/*.py`):

Monolithic — one call:

```
system = prompts/monolithic/system.txt
user   = manifest prompt (verbatim)
```

Agentive (Executor + Verifier, max 2 loops) — first executor call uses the manifest prompt verbatim; the verifier then receives:

```
Task:{question}\n\nCandidate answer:\n{candidate}
```

If the verifier does not reply starting with `APPROVE`, the executor is re-invoked with the templated revision message:

```
Your previous answer was:\n{candidate}\n\nVerifier feedback:
{feedback}\nProvide a corrected final answer.
```

Swarm — three parallel peers, each: peer system prompt + manifest prompt verbatim; the only difference between peers is the sampling seed (`seed = base_seed + peer_index`); answers are majority-voted.

Judge (secondary metric) — composed by `harness/judge.py` from the judge system prompt + the task, gold answer, and the recorded model answer.

Decoding pins (all calls): temperature 0.0, seed 42 (+peer offset), `num_ctx` 8192, `max_tokens` 6144 — from `config/config.yaml`, whose raw bytes are hashed into every row's `config_hash`.

4 Recipes

Print one task's exact user message (repo root, any machine):

```
./venv/bin/python -c "import json;
m=json.load(open('tasks/frontier_v2_manifest.json'));
print([t for t in m['tasks'] if t['task_id']=='fx2-mathA-001'][0]['prompt'])"
```

Dump every question to a file (e.g. for a report appendix):

```
./venv/bin/python -c "import json;
m=json.load(open('tasks/frontier_v2_manifest.json'));
print('\n\n'.join(f'=== {t['task_id']} ({t['source_id']}) ===\n{t['prompt']}'
for t in m['tasks']))"
> all_frontier_questions.txt
```

Audit what a historical run actually sent — join a result row back to its inputs via `task_id`, or read the row's own `raw_trace`, which records each agent call's user message and output:

```
./venv/bin/python -c "import json;
r=[json.loads(l) for l in open('results/frontier-v2-local-ollama-14b.jsonl')];
row=[x for x in r if x['task_id']=='fx2-mathA-001' and
x['backend']=='agentic'][0];
print(json.dumps(row['raw_trace'], indent=2))"
```

Byte-exact historical reproduction: every row carries `config_hash`; check out the commit where `config/config.yaml` hashes to that value (v1 freeze: tag `data-freeze-2026-07-01`) and the three layers above are guaranteed identical to what ran.

5 Worked example — fx2-mathA-001

Manifest fields (from `tasks/frontier_v2_manifest.json`):

```
task_id:  fx2-mathA-001
domain:  math  tier:  frontier
source_id:  aime25:0  (AIME 2025, problem index 0 in math-ai/aime25)
id_status:  FROZEN_CALIBRATED_IN_BAND
answer:  70
```

The `prompt` field — and therefore the exact user message, shown here verbatim including its LaTeX-style math notation, which is passed through as plain text:

```
Find the sum of all integer bases  $b > 9$  for which  $17_b$  is a divisor of
 $97_b$ .

Give the final answer as a single integer on the last line.
```

For a *monolithic* run, the model receives `prompts/monolithic/system.txt` as the system message and the text above as the user message, decoded at temperature 0.0, seed 42, output cap 6144 — nothing else. For *swarm*, three peers receive the identical pair with seeds 42/43/44. For *agentic*, the executor starts from the same pair and the verifier/revision templates of §3 take over from there — with the exact realized sequence stored in the run row's `raw_trace`.

Upstream provenance chain

`source_id` → fetch/verify tooling: baseline items are verified against their public sources by `scripts/verify_task_ids.py`; frontier items were built (and self-graded through the real grader) by `scripts/build_frontier_v2.py` from HuggingFace datasets (`math-ai/aime25`, `bigcode/bigcodebench-hard`, `dgslibisey/MuSiQue`). The full re-sourcing history and every deviation is in `PRE_REGISTRATION.md`'s Amendment Log.